SP
2025

Project 2: Carry Select Adder Design & Implementation

CE7325 AVLSI

MARK IVEY (MXI240020);LAMIN.JAMMEH@UTDALLAS.EDU & LAMIN JAMMEH (DAL852207)

UT DALLAS | ECE Department

Abstract:

In this project a Carry Select Adder (CSA) using ASAP7 nm technology with FinFET gates is designed. The CSA consists of two main parts an adder and a Carry_sel. The two work together to form a carry Select Adder.

HDL codes

The code consists of two parts one for the design module and one for the testbench to verify the design.

HDL design module

```
1 `default_nettype none
2
3 /////////////// Mirror Carry//////////////////////
4 module m_carry (
5 input wire a,
6 input wire b,
7 input wire c,
8 output wire c_out_n
9 );
10 assign c_out_n = ~((a & b) | ((a | b) & c));
11 endmodule
12
13 /////////////// Mirror Sum ////////////////////////
14 module m_sum (
15 input wire a,
16 input wire b,
17 input wire c_in,
18 input wire c_out_n,
19 output wire s_out
20 );
21 assign s_out = ~((c_in & a & b) | ((c_in | a | b) & c_out_n));
22 endmodule
23
24 /////////////// Four Bit Ripple Carry Adder ////////////////////
25 module four_bits (
26 input wire [3:0] i_a,
27 input wire [3:0] i_b,
28 input wire c_in,
29 output wire c_out,
30 output wire [3:0] o_sum
31 );
32 wire [3:0] a, b, sum;
33 wire [4:0] carry, n_carry;
34 genvar i;
35
36 generate
37 for (i = 0; i < 4; i = i + 1) begin
38 m_sum m_sum_0 (
```

```
39 .a(a[i]),
40 .b(b[i]),
41 .c_in(carry[i]),
42 .c_out_n(carry[i+1]),
43 .s_out(sum[i])
44 );
45
46 m_carry m_carry_0 (
47 .a(a[i]),
48 .b(b[i]),
49 .c(carry[i]),
50 .c_out_n(carry[i+1])
51 );
52
54 ////// not gates for even/odd bits /////
55 if (i % 2 == 0) begin
56 assign a[i] = i_a[i];
57 assign b[i] = i_b[i];
58 assign o_sum[i] = ~sum[i];
59 end else begin
60 assign a[i] = ~i_a[i];
61 assign b[i] = ~i_b[i];
62 assign o_sum[i] = sum[i];
63 end
64 end
65 endgenerate
66
67 assign c_out = carry[4];
68 assign carry[0] = c_in;
69 endmodule
70
71 ////////////////// Carry Select Adder ////////////////////////
72 module carry_sel (
73 input wire [12 - 1 : 0] i_a,
74 input wire [12 - 1 : 0] i_b,
75 input wire c_in,
76 output wire [12 - 1 : 0] o_sum,
77 output wire c_out
78 );
79
80 wire cs_3, cs_l, cs_u, c_out_l, c_out_u;
81 wire [11:4] sum_lb, sum_ub;
82 wire ub_cs;
83 assign c_out = (ub_cs) ? c_out_u : c_out_l; //bit 12, or c_out mux
84
85 ////////// Std ripple carry for bits 3:0
86 four_bits four_bits_3_0 (
87 .i_a (i_a[3:0]),
88 .i_b (i_b[3:0]),
89 .c_in (c_in),
90 .c_out(cs_3),
91 .o_sum(o_sum[3:0])
92 );
```

```
93
94 /////////// Bits 7:4 with carry-in/no carry-in ///////////
95 four_bits four_bits_7_4_L (
96 .i_a (i_a[7:4]),
97 .i_b (i_b[7:4]),
98 .c_in (1'b0),
99 .c_out(cs_l),
100 .o_sum(sum_lb[7:4])
101 );
102 four_bits four_bits_7_4_U (
103 .i_a (i_a[7:4]),
104 .i_b (i_b[7:4]),
105 .c_in (1'b1),
106 .c_out(cs_u),
107 .o_sum(sum_ub[7:4])
108 );
109
110 ////// Bits 11:8 with carry-in/no carry-in //////////////
111 four_bits four_bits_11_8_L (
112 .i_a (i_a[11:8]),
113 .i_b (i_b[11:8]),
114 .c_in (1'b0),
115 .c_out(c_out_l),
116 .o_sum(sum_lb[11:8])
117 );
118 four_bits four_bits_11_8_U (
119 .i_a (i_a[11:8]),
120 .i_b (i_b[11:8]),
121 .c_in (1'b1),
122 .c_out(c_out_u),
123 .o_sum(sum_ub[11:8])
124 );
125
126 assign ub_cs = (cs_3 & cs_u) | cs_l; // input logic for 11:8 output assignment mux
127 assign o_sum[7 : 4] = cs_3 ? sum_ub[7:4] : sum_lb[7:4]; // 7:4 mux
128 assign o_sum[11 : 8] = ub_cs ? sum_ub[11:8] : sum_lb[11:8]; // 11:8 mux
130 endmodule
```

Testbenches

Adder_testbench

```
2 module carry_sel_tb;
3
6 //Ports
7 reg [12 - 1 : 0] i_a;
8 reg [12 - 1 : 0] i_b;
9 reg c_in;
10 wire [12 : 0] o_sum;
11 wire c_out;
12 event start;
13 wire [12 : 0 ] s_out;
16 assign o_sum [12] = c_out;
17 carry_sel carry_sel_inst (
18 .i_a(i_a),
```

```
19 .i_b(i_b),
20 .c_in(c_in),
21 .o_sum(o_sum),
22 .c_out(c_out)
23 );
24 bit [24:0] jj;
27 initial begin
28 i_a = 'x;
29 i_b = 'x;
30 c_in = 'x;
31 #1;
32 $display("|\t\tCount\t|\t a\t\t|\t b\t\t|\t c\t|\t sum\t|");
33 $display("| %9d \t| %4h \t| %4h \t| %2h \t| %5h \t|", jj, i_a, i_b, c_in, {
c_out, o_sum});
34 for (int ii = 0; ii < 2**25; ii++) begin
35 i_a = ii[11:0];
36 i_b = ii[23:12];
37 c_in = ii[24];
38 #1;
39
40 assert ({c_out, o_sum} == (jj[11:0] + jj[23:12] + jj[24]))
41 else $fatal("ERROR!!!!\t\t\t a\t %0d b\t %0d c_in\t %0d sum \t %0d \t carry \t
%0d\n Expected: %0d",
42 i_a, i_b, c_in, {c_out, o_sum}, c_out, (jj[11:0] + jj[23:12] + jj[24]));
43
44 if (jj % 234561 == 0)
45 $display("| %9d \t| %4d \t| %4d \t| %2d \t| %5d \t|", jj, i_a, i_b, c_in, {
c_out, o_sum});
46 jj++;
47 end
48 -> start;
49
50 ////////////////////////////////////////////////
51 //Phase 2 start
52 // No need to assert the output as it was already
53 // tested in Phase 1
54 ////////////////////////////////////////////////
55
56 ////////////////////////////////////////////////
57 // 0x0 (a) + 0x0 (b) + 0x0 (c_in) = 0x0 ({c_out, s})
58 ////////////////////////////////////////////////
59 i_a = '0;
60 i_b = '0;
61 c_in = '0;
62 #5;
63
64 ////////////////////////////////////////////////
65 // 0xFFF (a) + 0x0 (b) + 0x0 (c_in) = 0x0FFF ({c_out, s})
66 ////////////////////////////////////////////////
67 i_a = 'hFFF;
68 i_b = '0;
69 c_in = '0;
70 #5;
```

```
72 ////////////////////////////////////////////////
73 // 0xFFF (a) + 0x0 (b) + 0x1 (c_in) = 0x1000 ({c_out, s})
74 ////////////////////////////////////////////////
75 i_a = 'hFFF;
76 i_b = '0;
77 c_in = 'b1;
78 #5;
79
80 ////////////////////////////////////////////////
81 // 0x420 (a) + 0xFFF (b) + 0x1 (c_in) = 0x142F ({c_out, s})
82 ////////////////////////////////////////////////
83 i_a = 'h420;
84 i_b = 'hFFF;
85 c_in = '0;
86 #5;
87
88 ////////////////////////////////////////////////
89 // 0xFFF (a) + 0xFFF (b) + 0x0 (c_in) = 0x1FFE({c_out, s})
90 ////////////////////////////////////////////////
91 i_a = 'hFFF;
92 i_b = 'hFFF;
93 c_in = '0;
94 #5;
96
97 ////////////////////////////////////////////////
98 // 0xFFF (a) + 0xFFF (b) + 0x1 (c_in) = 0x1FFF({c_out, s})
99 ////////////////////////////////////////////////
100 i_a = 'hFFF;
101 i_b = 'hFFF;
102 c_in = 'b1;
103 #5;
106 end
107
108 initial begin
109 $dumpfile("dump.vcd");
110 $dumpvars(2, carry_sel_tb);
111 $dumpvars(2, carry_sel);
112 $dumpoff;
113
114 @(start);
115 $dumpon;
116
117 #400
118 $finish;
119 end
121 endmodule
```

4_bits_testbench

```
1
2 module four_bits_tb;
6 //Ports
7 reg [3:0] i_a;
```

```
reg [3:0] i_b;
reg c_in;
wire c_out;
wire [3:0] o_sum;

four_bits four_bits_inst (
.i_a(i_a),
.i_b(i_b),
.c_in(c_in),
.c_out(c_out),
.o_sum(o_sum)
);

bit [8:0] jj;

//always #5 clk = ! clk ;
initial begin
i_a = 0;
i_b = 0;
c_in = 0;
#1;
$display("| Count\t\t| a\t| b\t| c\t| sum\t|");
$monitor("| %3d \t| %2d \t| %2d \t| %2d \t| %2d \t|", jj, i_a, i_b, c_in, {
c_out, o_sum});
for (int ii = 0; ii < 512; ii++) begin
i_a = ii[3:0];
i_b = ii[7:4];
c_in = ii[8];
#1;

assert ({c_out, o_sum} == (jj[3:0] + jj[7:4] + jj[8]))
else $error("ERROR!!!!\t\t\t a\t %0d b\t %0d c_in\t %0d sum \t %0d \t carry \t
%0d\r\n Expected: %0d",
i_a, i_b, c_in, {c_out, o_sum}, c_out, (jj[3:0] + jj[7:4] + jj[8]));

jj++;
end
// $display ("sum \t %0b \t carry \t %0b", o_sum, c_out);
end

initial begin
$dumpfile("dump.vcd");
$dumpvars(2, four_bits_tb);
$dumpvars(2, four_bits);

#600
$finish;
end

endmodule
```

HDL output pre synthesis

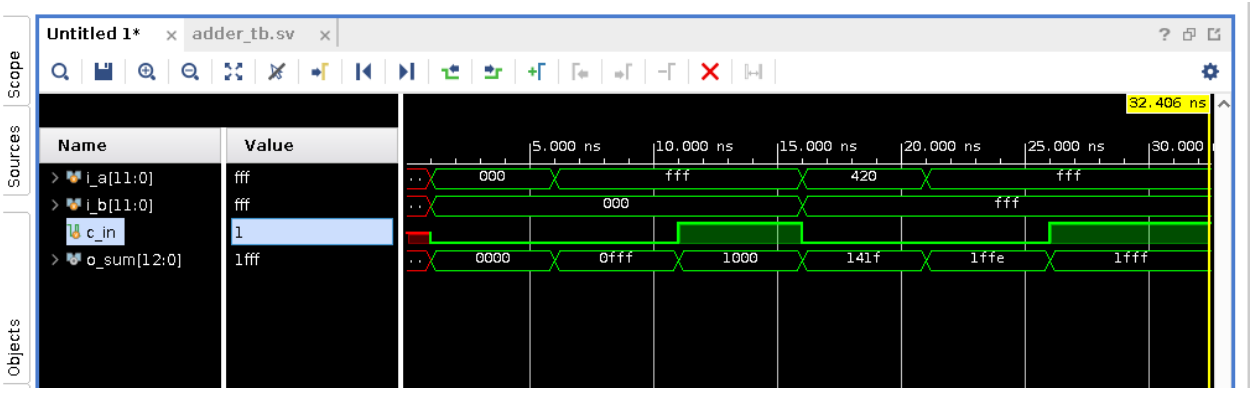

Figure1: HDL output pre-synthesis

Schematic

The design logic behind the creation of this 12 bit RCA is as follows:

1) First, the logic cells mirror carry and Mirror sum were created
2) Next, a 4-bit adder was created using those cells
3) After that, a carry select block was created using the 4-bit adder and mux cells
4) Finally, one 4-bit adder and two carry select blocks were combined in the final schematic to create the CSA.
5)

The design uses a total of 90 cells, which use the following gates from project 1:

1) Not
2) Mux 2_1
3) NOR2
4) NAND2

As well as the newly created M_SUM and M_CARRY gates.

Mirror Sum

$$Si = A_iB_iC_i + (\sim C_{i+1})(A_i + B_i + C_i)$$

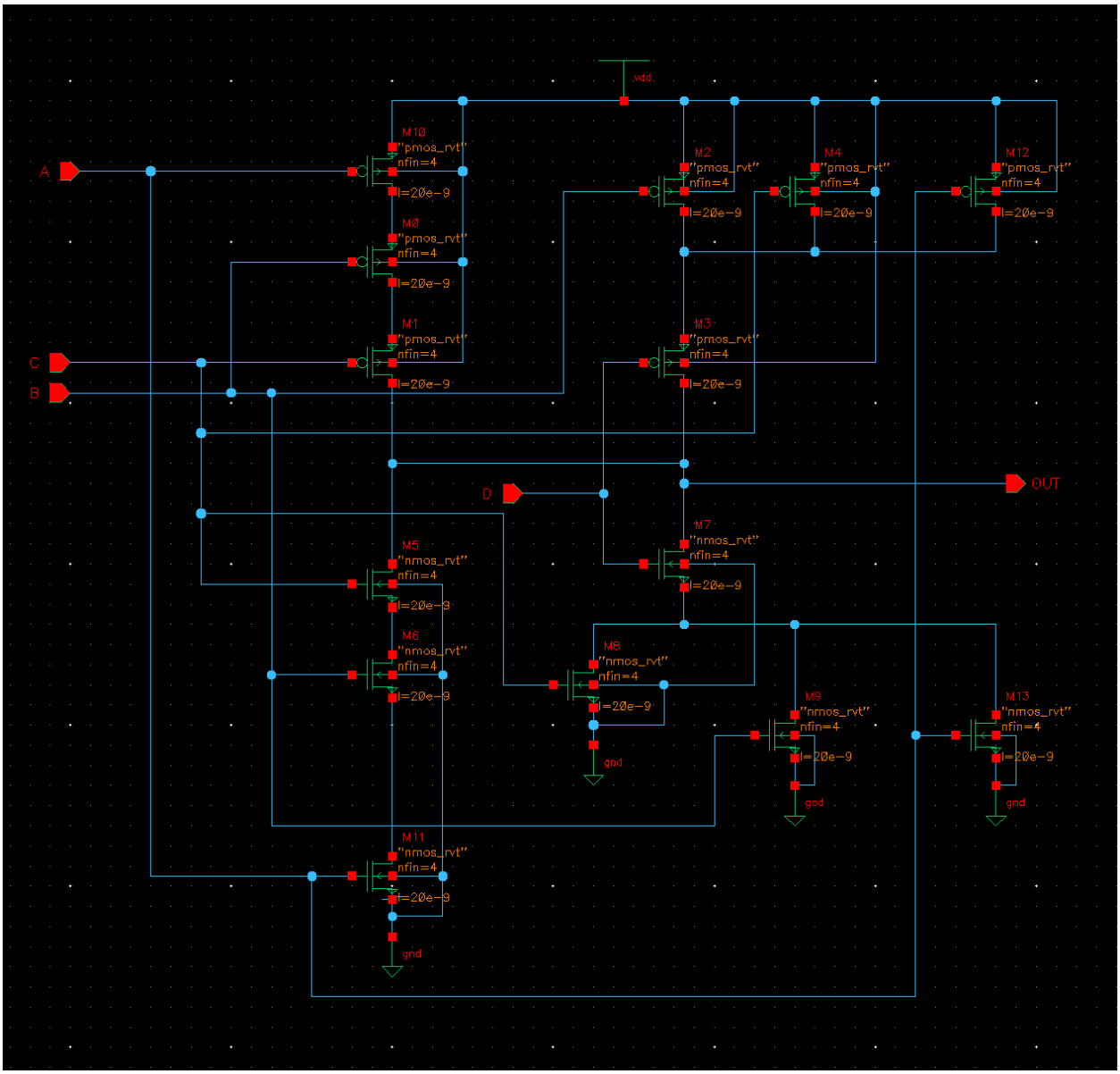

Figure2: Mirror Sum

Mirror Carry

$$C_{i+1} = A_i B_i + C_i(A_i + B_i)$$

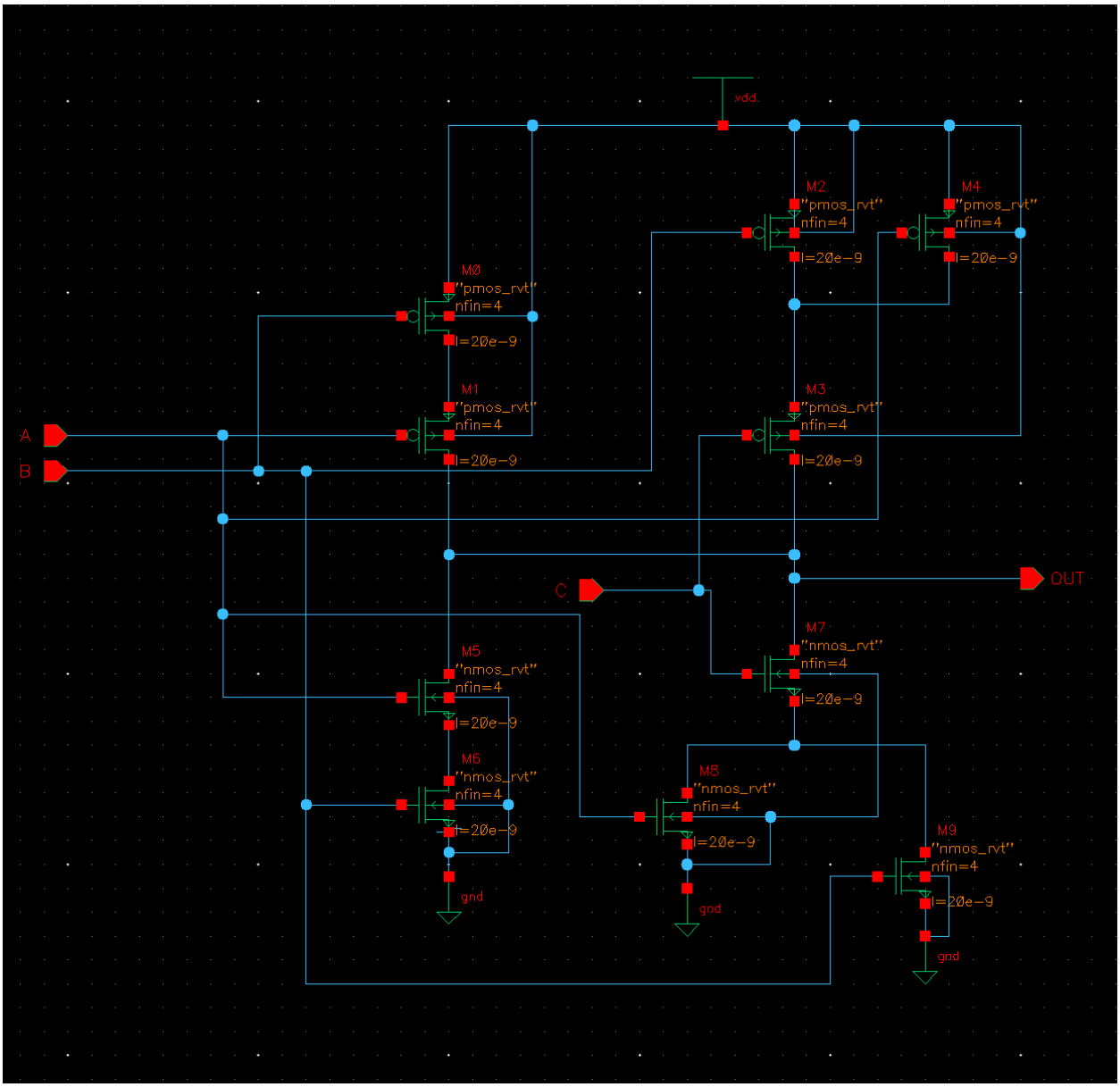

Figure3: Mirror sum

Four-Bit Adder

This is the Adder, it is made using the mirror carry, mirror sum and inverters on the even outputs, and odd inputs, as shown in class.

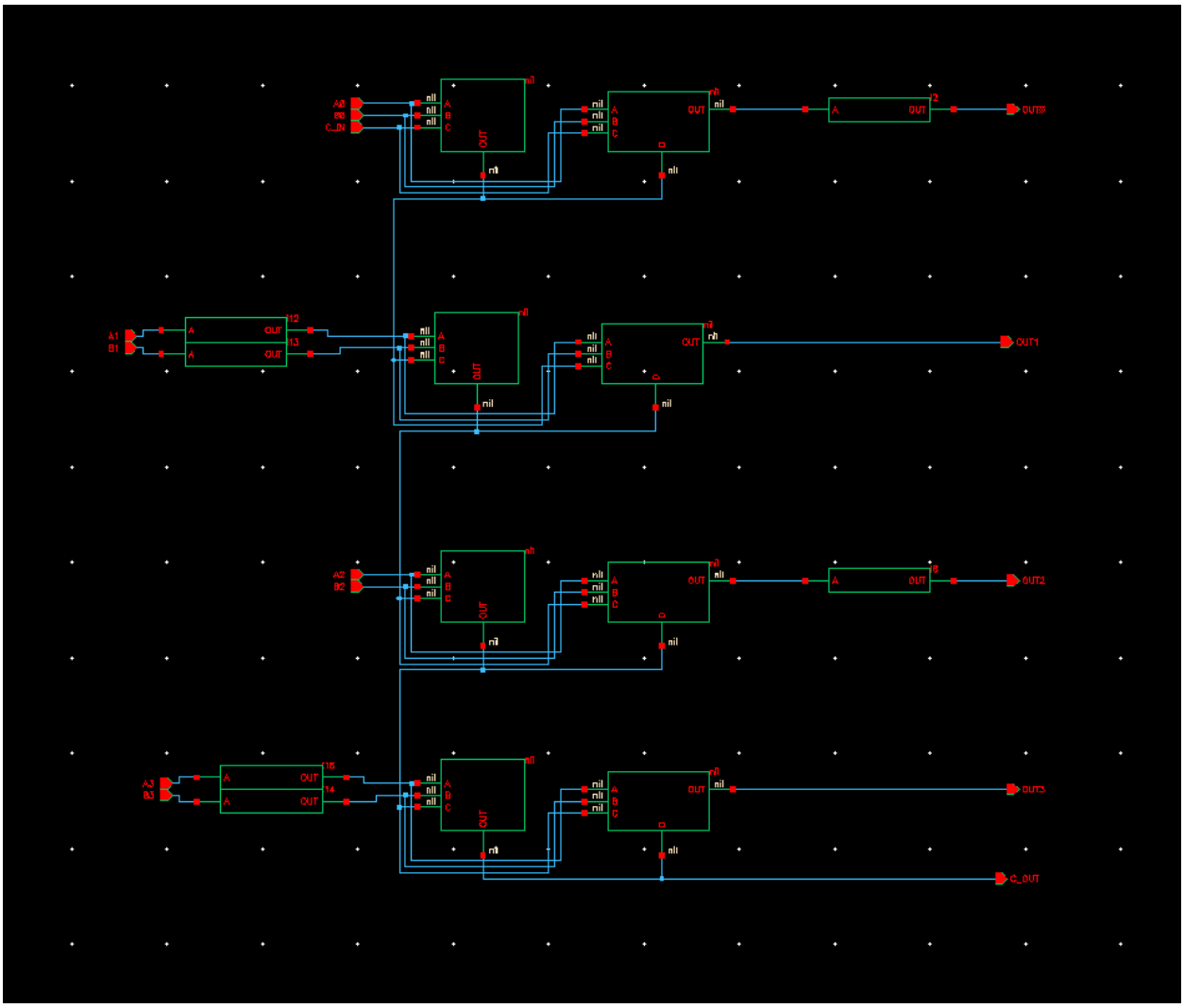

Figure4: Four Bit Adder

Four-Bit Carry Select

Using the adder mentioned in figure 4, we combine two adders and five 2:1 muxes to get the output and the carry. One adder has its carry-in input tied to ground, and the other, tied to VDD as discussed in class. The inputs to the 2:1 mux from the carry-in used to select which adder results to use needed a buffer and fanout to minimize gate delay.

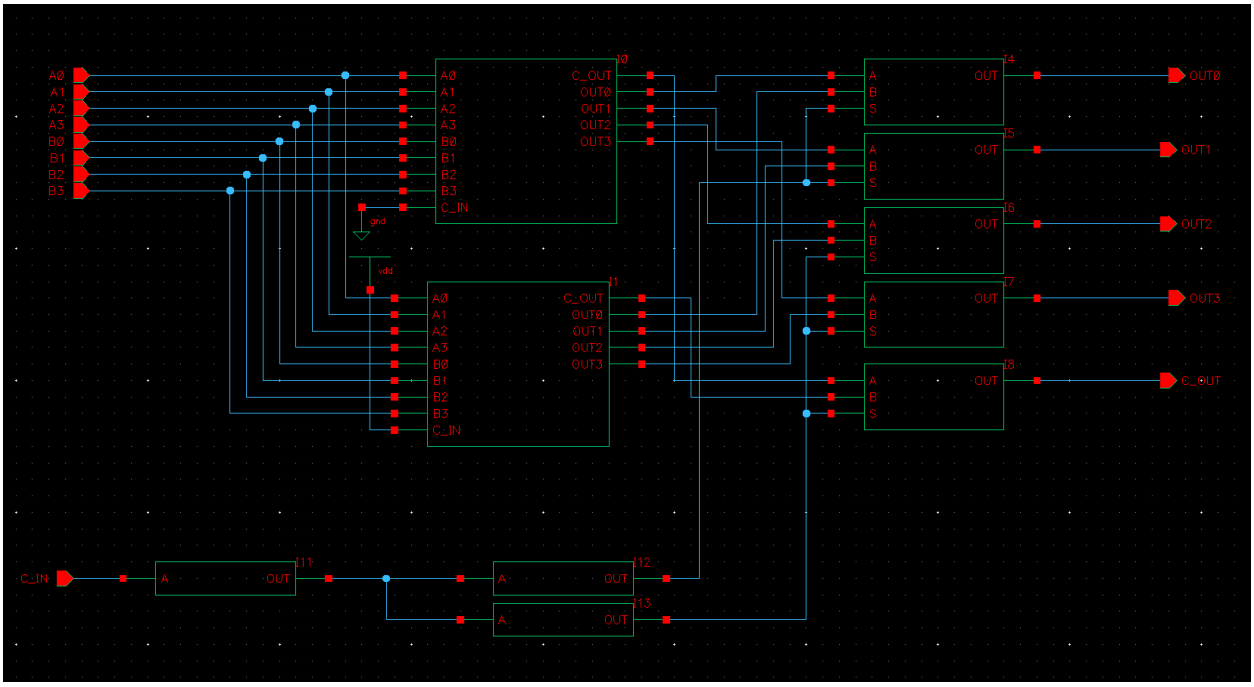

Figure5: Four-bit carry select

CSA Block

Using an adder from figure 4 and two carry select blocks as shown in figure 5, we chain the carry inputs and outputs together to create the 12-bit CSA. With more time, a separate CSA block would have been made for the third block, using NAND2 and NOR2 gates to possibly minimize the complexity, but due to time constrains, said gates were implemented as not gates to meet the four gate from project 1 requirement.

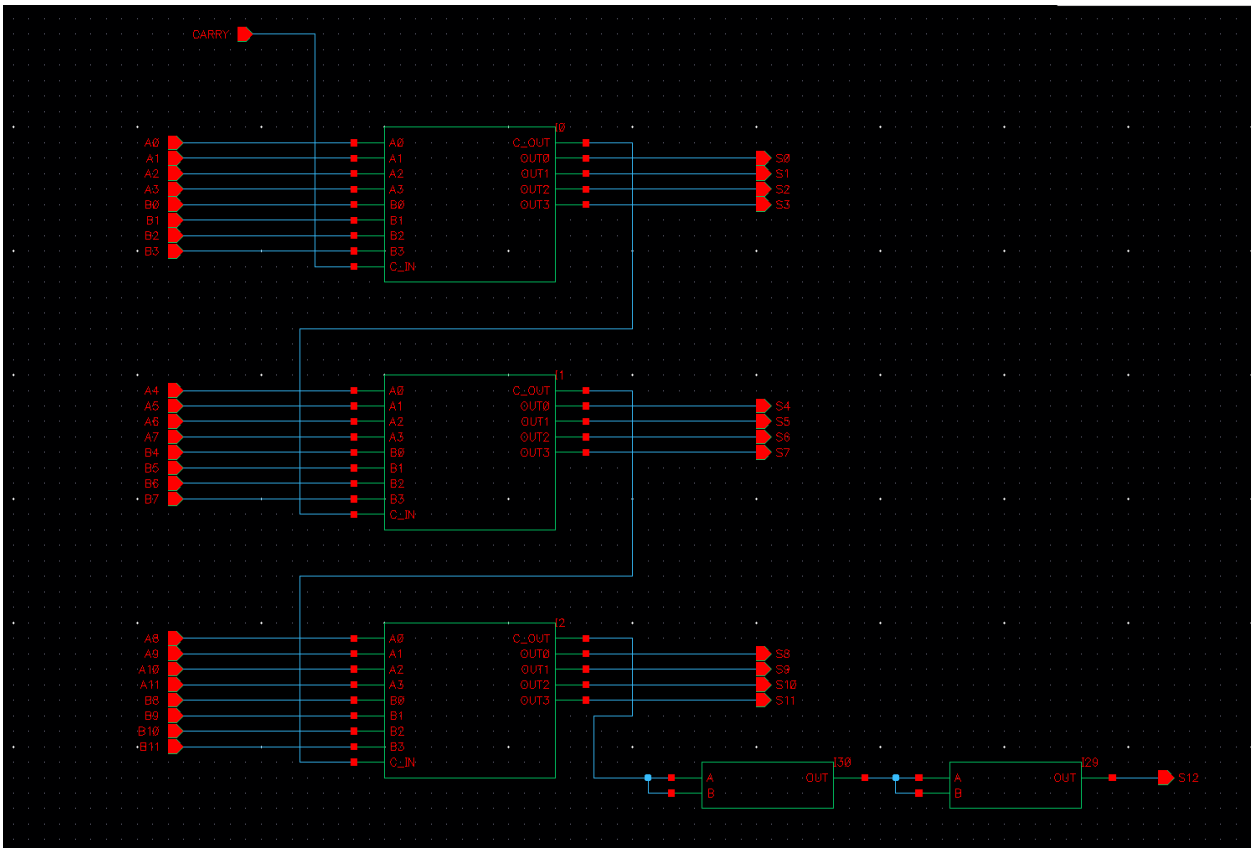

Figure6: CSA block diagram

Layout

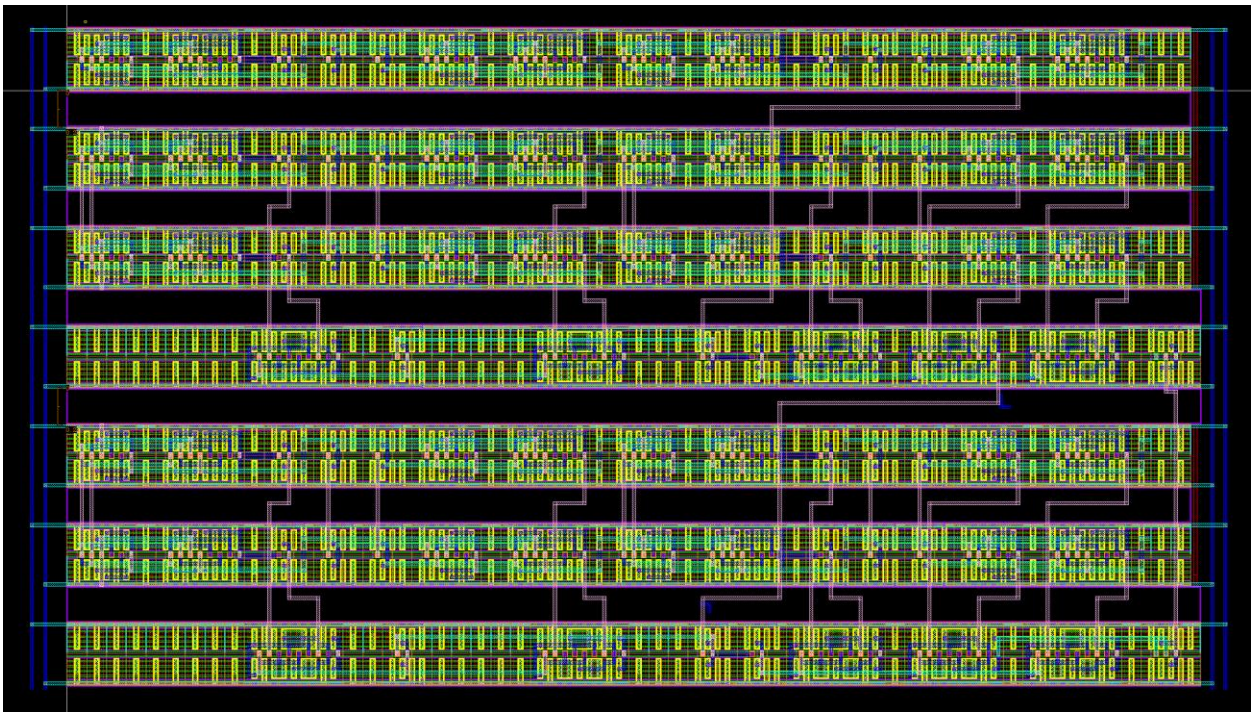

Figure7: layout of 12_bit_CSA from Cadence Virtuoso

DRC results of the 12_bit_CSA

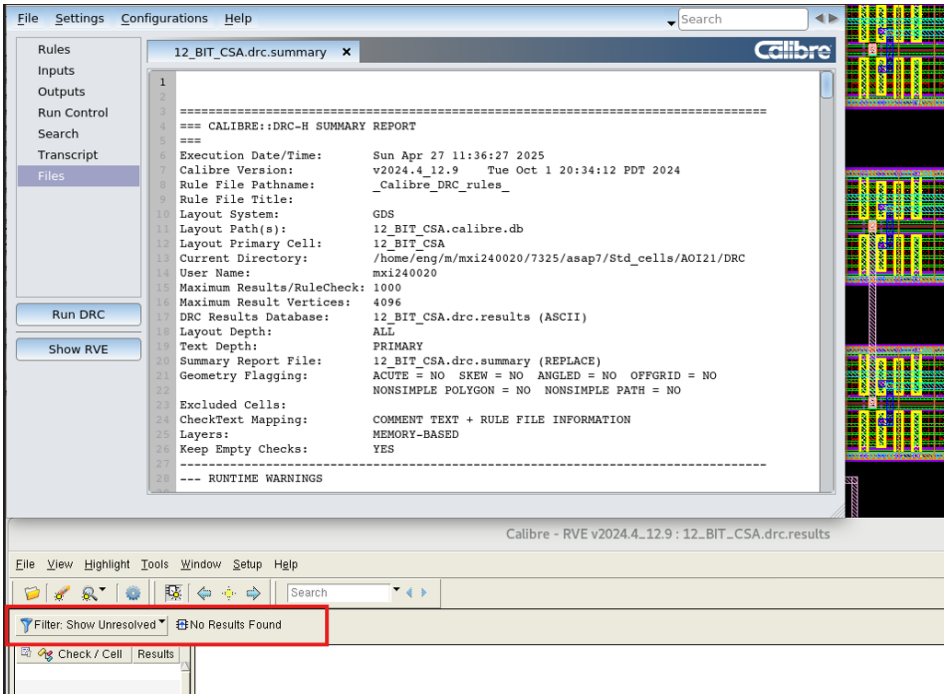

Figure8: DRC results of the 12_bit_CSA

LVS

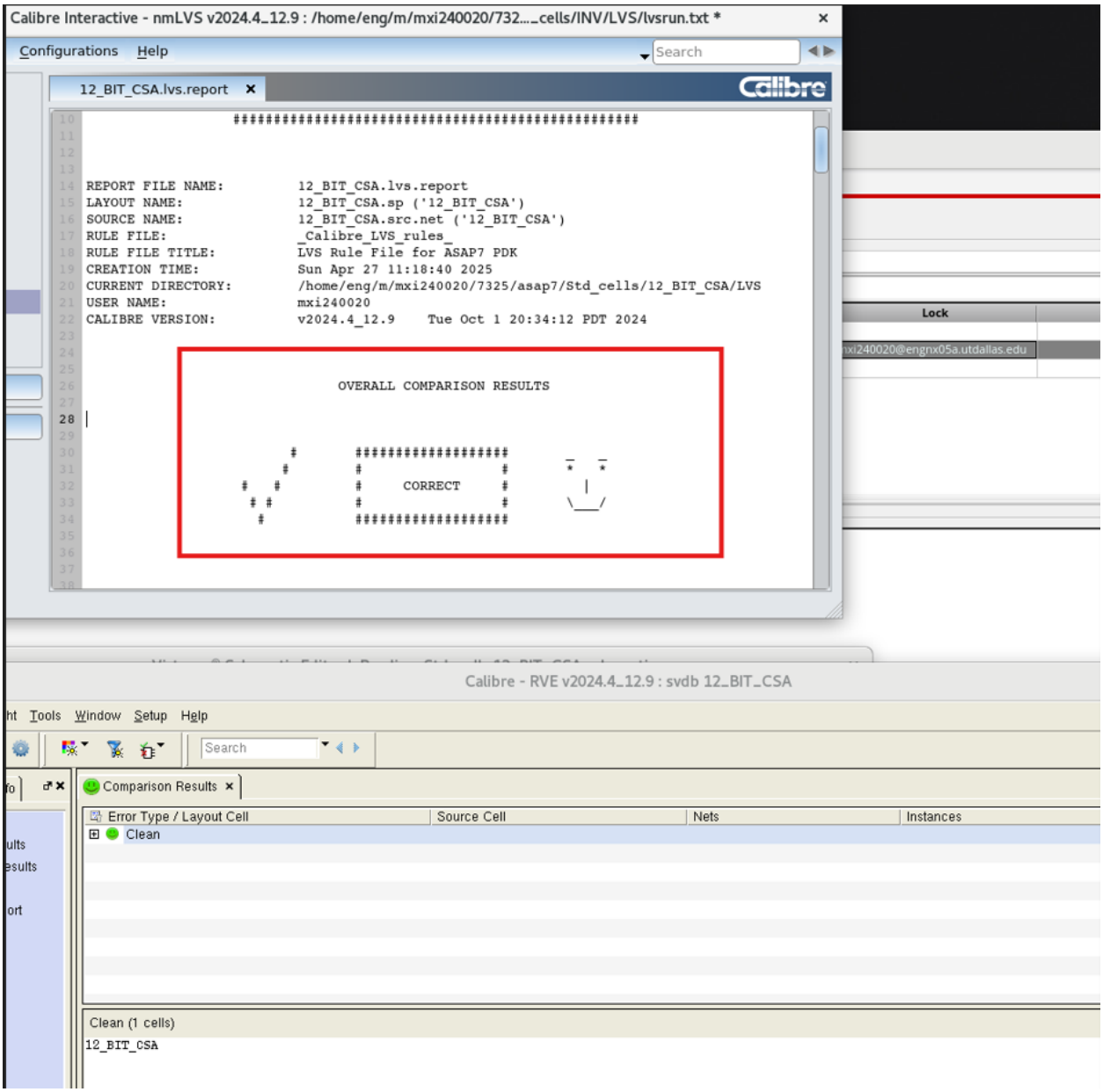

Figure9: LVS results of the 12 bit CSA

Post PnR simulation and HSPICE analysis:

Part 1 of the simulation:

Input waveform for A_0 to A_{11}

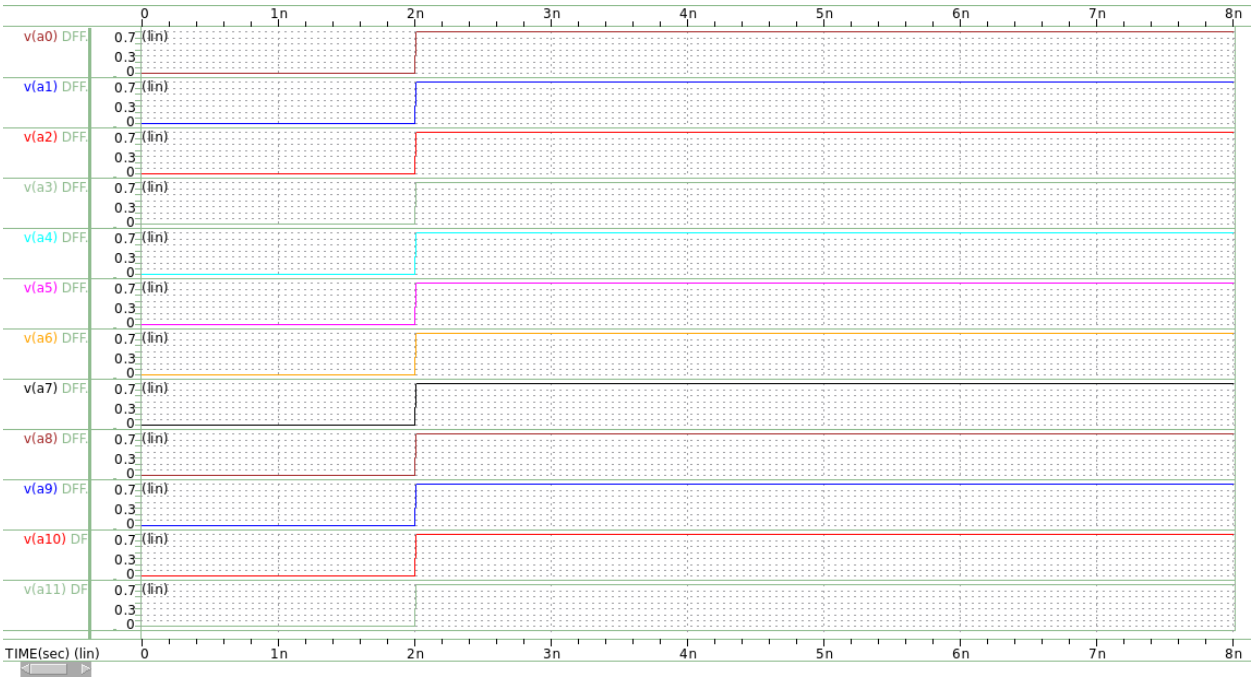

Figure10: showing waveforms for input A

Input waveform for B_0 to B_{11} and C

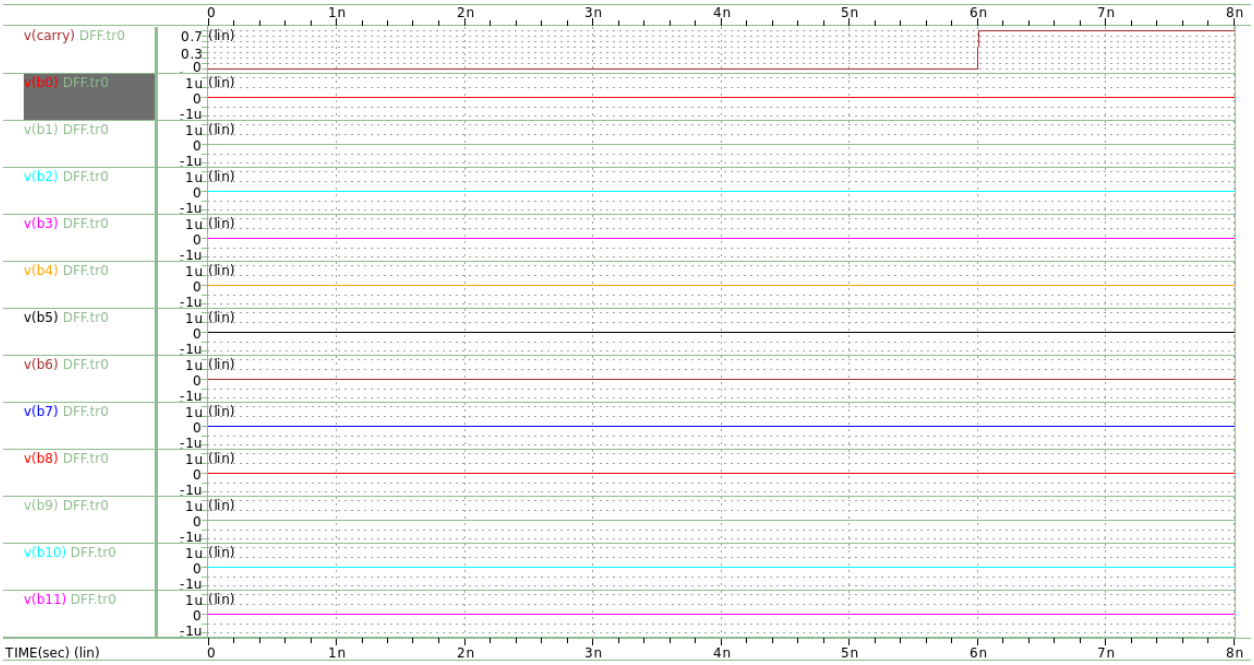

Figure11: showing waveforms for input B and C

Output waveform for part1 post synthesis

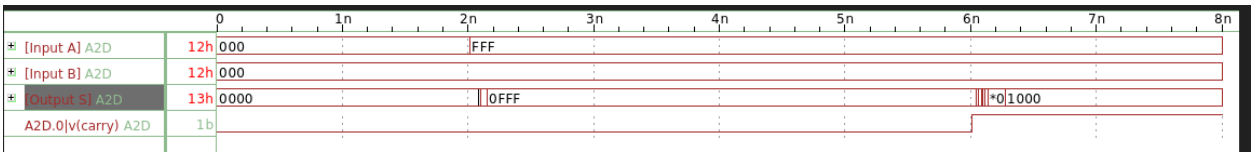

Figure12: showing the netlist simulation

Part 1 HSPICE analysis:

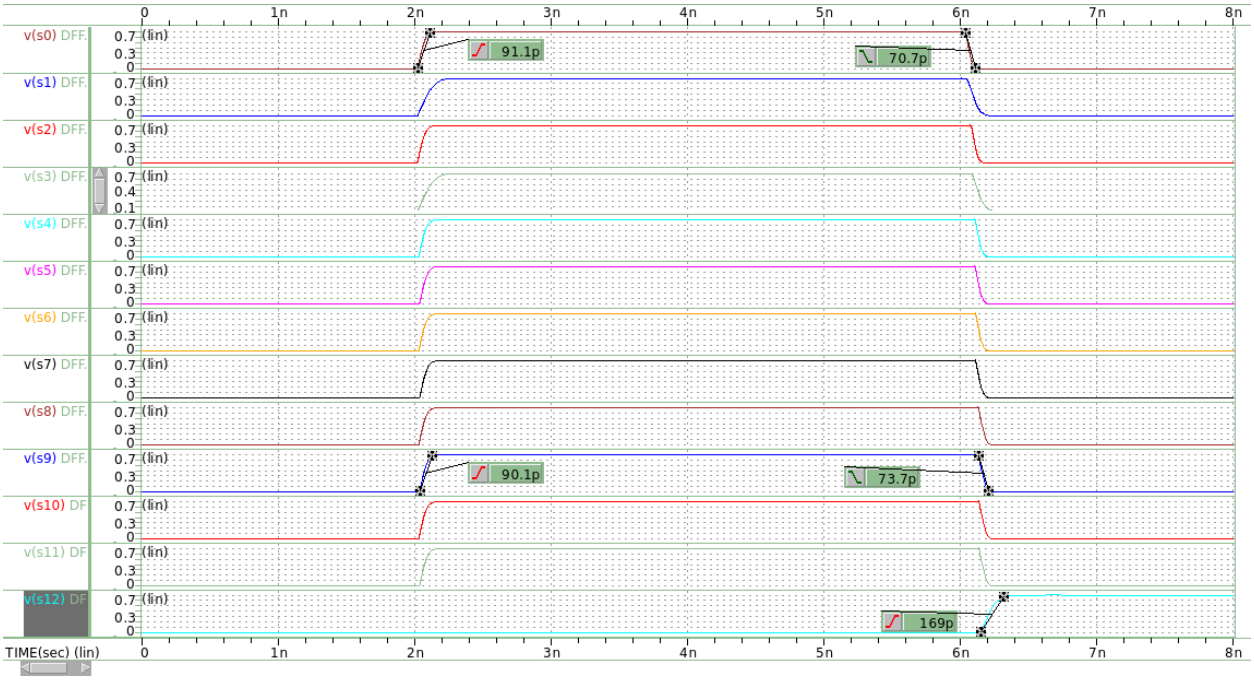

Figure13: showing the HSPICE rise times

Part 2 of the simulation:

Input waveform for A_0 to A_{11}

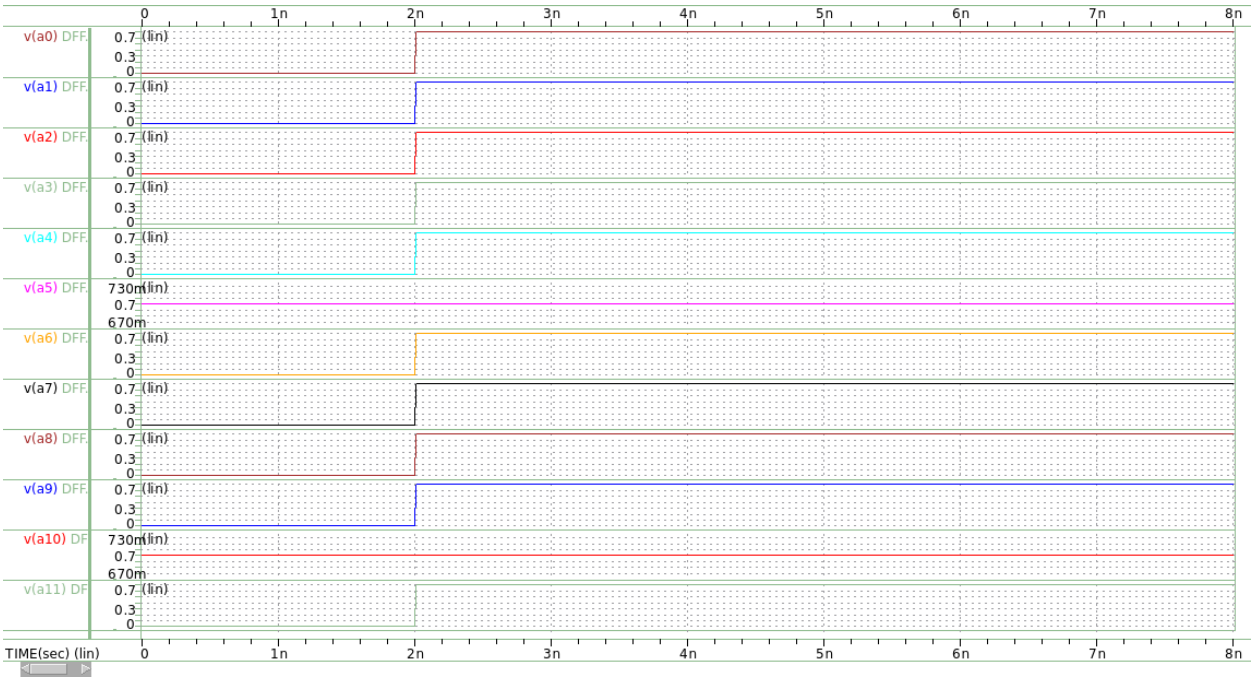

Figure14: showing waveforms for input A

Input waveform for B_0 to B_{11} and C

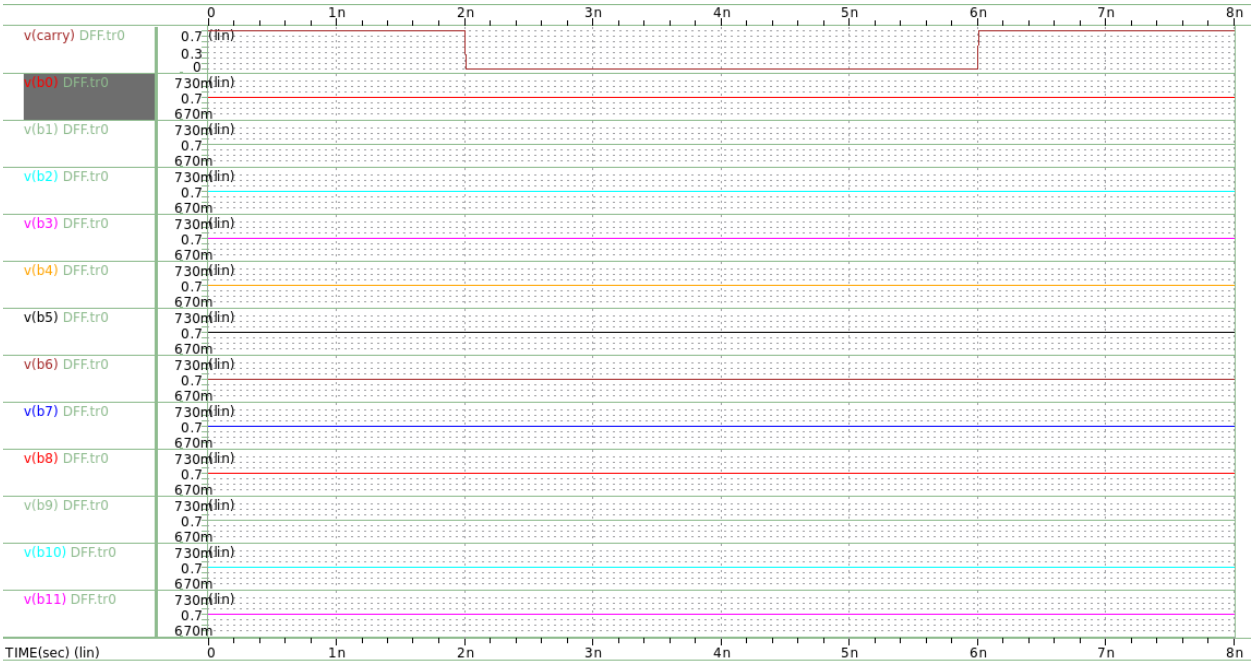

Figure15: showing waveforms for input B and C

Output waveform for part2 post synthesis

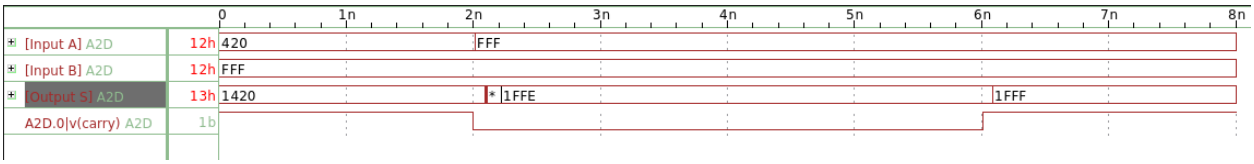

Figure16: showing part2 of the netlist simulation

Part 2 HSPICE analysis:

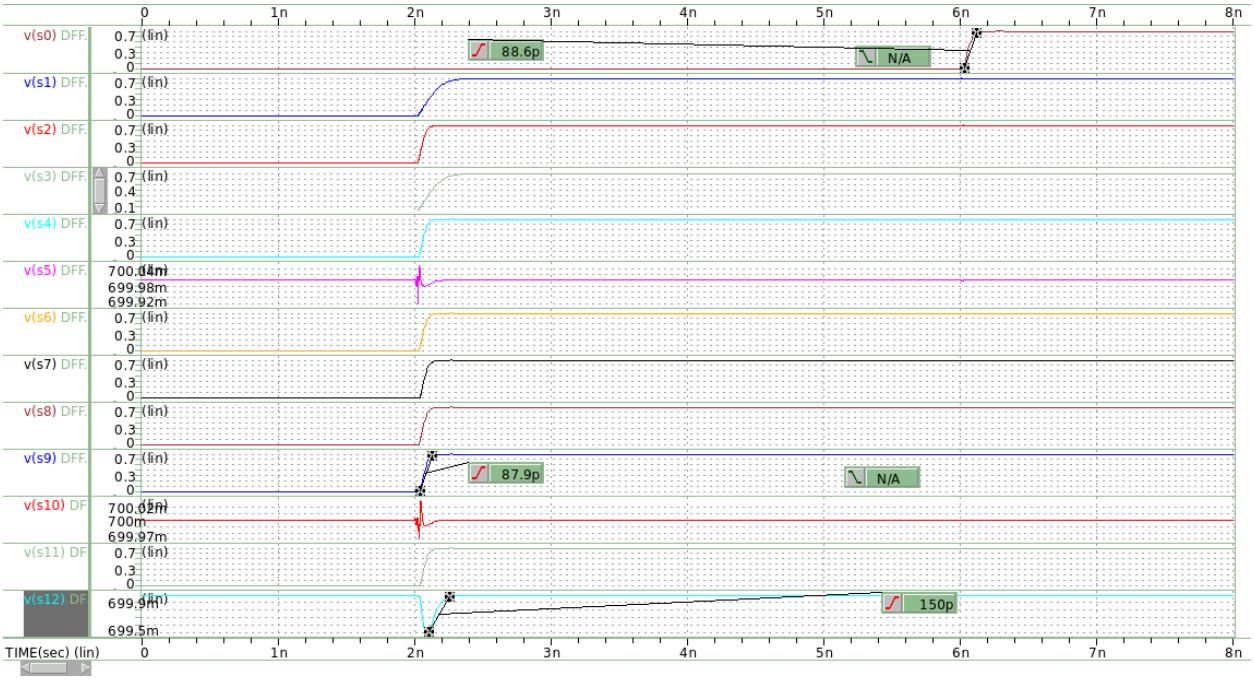

Figure17: showing the HSPICE rise times

Conclusion and Final Thoughts

We were successfully able to implement the Carry select adder as discussed in class. The schematic was created using a series of repeating blocks built and tested one module at a time, (not shown due to time and report length constraints, but included in the zip file). The output waveforms shown on figures 16 and 12 do in fact combine to show the output tests done on figure 1 for logic simulation.

While we were able to test and provide the report, Hspice, LVS, DRC and schematic as required by the project, it is somewhat regrettable that we were unable to use tools to do STA, or auto place and route using Innovus. I think additional testing would be needed to try and find the worst-case scenario, and with HSPICE, that would take to long and possibly consume too much resources.